

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



ViewModel dan Debugging

Oleh:

Akhmad Chaidar Ananda

NIM. 2310817110015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel dan Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akhmad Chaidar Ananda
NIM : 2310817110015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
SOAL 1	6
A. Source Code	6
B. Output Program	39
C. Pembahasan	43
SOAL 2	58
A. Pembahasan	58
Tautan Git.....	59

DAFTAR GAMBAR

Gambar 1. Screenshot Output Soal 1	39
Gambar 2. Screenshot Output Soal 1	40
Gambar 3. Screenshot Output Soal 1	41
Gambar 4. Screenshot Output Soal 1	42
Gambar 5. Screenshot Hasil Debugging	43
Gambar 6. Screenshot Hasil Debugging	43

DAFTAR TABEL

Tabel 1. Source Code Jawaban Soal 1	6
Tabel 2. Source Code Jawaban Soal 1	7
Tabel 3. Source Code Jawaban Soal 1	11
Tabel 4. Source Code Jawaban Soal 1	13
Tabel 5. Source Code Jawaban Soal 1	15
Tabel 6. Source Code Jawaban Soal 1	17
Tabel 7. Source Code Jawaban Soal 1	17
Tabel 8. Source Code Jawaban Soal 1	18
Tabel 9. Source Code Jawaban Soal 1	19
Tabel 10. Source Code Jawaban Soal 1	22
Tabel 11. Source Code Jawaban Soal	25
Tabel 12. Source Code Jawaban Soal 1	28
Tabel 13. Source Code Jawaban Soal 1	32
Tabel 14. Source Code Jawaban Soal 1	34
Tabel 15. Source Code Jawaban Soal 1	35
Tabel 16. Source Code Jawaban Soal 1	38

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory dalam pembuatan ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

1. character.kt

Tabel 1. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.mode
3	ls
4	
5	import android.os.Parcelable
6	import kotlinx.parcelize.Parcelize
7	
8	@Parcelize
9	data class Character (
10	val name: String,

11	val warikName: String,
12	val linkInstagram: String,
13	val photo: Int,
14	val details: String
15	): Parcelable
16	
17	
18	
19	
20	

2. item_character.xml

Tabel 2. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res
4	/android"
5	
6	xmlns:tools="http://schemas.android.com/tools"
7	android:layout_width="match_parent"
8	android:layout_height="wrap_content"
9	
10	xmlns:app="http://schemas.android.com/apk/res-
11	auto"
12	android:id="@+id/card_view"
13	android:layout_gravity="center"
14	android:layout_marginStart="8dp"
15	android:layout_marginTop="4dp"
16	android:layout_marginEnd="8dp"
17	android:layout_marginBottom="4dp"
18	app:cardCornerRadius="4dp">

19
20

```
<androidx.constraintlayout.widget.ConstraintLayout  
t  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:padding="5dp"  
    android:background="@color/primary">  
  
    <ImageView  
        android:id="@+id/img_item_photo"  
        android:layout_width="70dp"  
        android:layout_height="85dp"  
        android:scaleType="centerCrop"  
  
    app:layout_constraintStart_toStartOf="parent"  
  
    app:layout_constraintBottom_toBottomOf="parent"  
  
    app:layout_constraintTop_toTopOf="parent"  
        android:src="@drawable/chaidar"  
    />  
  
    <TextView  
        android:id="@+id/tv_item_name"  
        android:layout_width="wrap_content"  
        android:layout_height="wrap_content"  
        android:text="Nama Char"  
        android:textColor="@color/text2"  
        android:textSize="16sp"  
        android:textStyle="bold"
```


	<pre> app:layout_constraintEnd_toEndOf="parent" app:layout_constraintHorizontal_bias="0.093" app:layout_constraintStart_toEndOf="@id/img_item_ photo" tools:layout_editor_absoluteY="3dp" /> <TextView android:id="@+id/tv_item_warik_name" android:layout_width="200dp" android:layout_height="wrap_content" android:layout_marginTop="3dp" android:text="Warik Name" android:textColor="@color/accent_color2" android:textSize="12sp" android:textStyle="italic" app:layout_constraintEnd_toEndOf="parent" app:layout_constraintHorizontal_bias="0.191" app:layout_constraintStart_toEndOf="@id/img_item_ photo" app:layout_constraintTop_toBottomOf="@id/tv_item_ name" /> <com.google.android.material.button.MaterialButto </pre>
--	--

	<pre> n android:id="@+id/btn_details" style="@style/Widget.Material3.Button" android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginStart="20dp" android:layout_marginTop="36dp" android:backgroundTint="@color/accent_color1" android:minWidth="0dp" android:minHeight="0dp" android:padding="6dp" android:text="@string/btn_detail" android:textAllCaps="true" android:textColor="@color/text2" android:textSize="10sp" app:layout_constraintStart_toEndOf="@id/img_item_ photo" app:layout_constraintTop_toBottomOf="@id/tv_item_ name" /> <com.google.android.material.button.MaterialButto n android:id="@+id/btn_goto_instagram" style="@style/Widget.Material3.Button" android:layout_width="wrap_content" android:layout_height="wrap_content" </pre>
--	--

	<pre> android:layout_marginStart="16dp" android:minWidth="0dp" android:minHeight="0dp" android:padding="6dp" android:text="Contact" android:textAllCaps="true" android:textSize="10sp" app:layout_constraintBottom_toBottomOf="parent" app:layout_constraintStart_toEndOf="@id/btn_details" android:textColor="@color/text2" android:backgroundTint="@color/accent_color1" /> </androidx.constraintlayout.widget.ConstraintLayout> </androidx.cardview.widget.CardView> </pre>
--	---

3. fragment_home.xml

Tabel 3. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	t
4	xmlns:android="http://schemas.android.com/apk/res
5	/android"

6	
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_width="match_parent"
9	android:layout_height="match_parent"
10	
11	xmlns:app="http://schemas.android.com/apk/res-
12	auto"
13	
14	tools:context=".presentation.home.HomeFragment"
15	android:background="@color/background">
16	
17	<androidx.recyclerview.widget.RecyclerView
18	android:id="@+id/rc_character"
19	android:layout_width="0dp"
20	android:layout_height="0dp"
	android:paddingTop="15dp"
	android:layout_margin="15dp"
	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintBottom_toBottomOf="parent"
	app:layout_constraintTop_toTopOf="parent"
	app:layout_constraintEnd_toEndOf="parent"
	tools:listitem="@layout/item_character"
	/>
	</androidx.constraintlayout.widget.ConstraintLayout>

4. fragment_detail.xml

Tabel 4. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	t
4	xmlns:android="http://schemas.android.com/apk/res
5	/android"
6	
7	xmlns:tools="http://schemas.android.com/tools"
8	android:layout_width="match_parent"
9	android:layout_height="match_parent"
10	
11	xmlns:app="http://schemas.android.com/apk/res-
12	auto"
13	
14	tools:context=".presentation.detail.DetailFragmen
15	t"
16	android:background="@color/background">
17	
18	<ImageView
19	android:id="@+id/img_item_photo"
20	android:layout_width="120dp"
	android:layout_height="120dp"
	android:layout_marginTop="84dp"
	android:scaleType="centerCrop"
	app:layout_constraintEnd_toEndOf="parent"
	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintTop_toTopOf="parent"
	android:src="@drawable/chaidar"
	/>

	<pre> <TextView android:id="@+id/tv_name" android:layout_width="wrap_content" android:layout_height="wrap_content" android:layout_marginTop="20dp" app:layout_constraintTop_toBottomOf="@id/img_item _photo" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" android:text="Nama Character" android:textSize="20sp" android:textStyle="bold" android:textColor="@color/text2" /> <TextView android:id="@+id/tv_warik_name" android:layout_width="wrap_content" android:layout_height="wrap_content" android:text="Warik..." android:textStyle="bold" android:textColor="@color/accent_color2" app:layout_constraintTop_toBottomOf="@id/tv_name" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" /> <TextView </pre>
--	---

	<pre> android:id="@+id/tv_details" android:layout_width="300dp" android:layout_height="wrap_content" android:text="Detail Warik" android:textStyle="bold" android:textColor="@color/accent_color2" app:layout_constraintTop_toBottomOf="@id/tv_warik_name" app:layout_constraintStart_toStartOf="parent" app:layout_constraintEnd_toEndOf="parent" /> </androidx.constraintlayout.widget.ConstraintLayout> </pre>
--	---

5. fragment_info.xml

Tabel 5. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res
4	/android"
5	
6	xmlns:tools="http://schemas.android.com/tools"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	
10	tools:context=".presentation.info.InfoFragment"
11	android:orientation="vertical"
12	android:gravity="center"
13	android:background="@color/background">

14	
15	<TextView
16	android:id="@+id/tv_item_title"
17	android:layout_width="wrap_content"
18	android:layout_height="wrap_content"
19	android:gravity="center"
20	android:text="@string/title"
	android:textStyle="bold"
	android:textSize="40sp"
	android:textColor="@color/text2"
	/>
	<ImageView
	android:id="@+id/img_item_photo_warik"
	android:layout_width="300dp"
	android:layout_height="300dp"
	android:src="@drawable/img"
	/>
	<TextView
	android:id="@+id/tv_item_description"
	android:layout_width="300dp"
	android:layout_height="wrap_content"
	android:text="@string/warik_information"
	android:textAlignment="textStart"
	android:layout_marginTop="8dp"
	android:textStyle="italic"
	android:textColor="@color/accent_color2"
	/>
	</LinearLayout>

6. bottom_navigation_menu.xml

Tabel 6. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<menu
3	xmlns:android="http://schemas.android.com/apk/res
4	/android">
5	<item
6	android:id="@+id/navigation_home"
7	android:title="Home"
8	android:icon="@drawable/ic_home_24"/>
9	
10	<item
11	android:id="@+id/navigation_info"
12	android:title="Info"
13	android:icon="@drawable/ic_info_24"
14	/>
15	</menu>

7. navigation.xml

Tabel 7. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	xmlns:android="http://schemas.android.com/apk/res
4	/android"
5	
6	xmlns:app="http://schemas.android.com/apk/res-
7	auto"
8	
9	xmlns:tools="http://schemas.android.com/tools"
10	android:id="@+id/navigation"
11	app:startDestination="@layout/fragment_home">

12	
13	<fragment
14	android:id="@+id/navigation_home"
15	
16	android:name="com.example.modul3scrollablelistwar
17	ikmaxxing.presentation.home.HomeFragment"
18	android:label="Home"
19	tools:layout="@layout/fragment_home"/>
20	
	<fragment
	android:id="@+id/navigation_info"
	android:name="com.example.modul3scrollablelistwar
	ikmaxxing.presentation.info.InfoFragment"
	android:label="Info"
	tools:layout="@layout/fragment_info"/>
	</navigation>

8. activity_main.xml

Tabel 8. Source Code Jawaban Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<RelativeLayout
3	xmlns:android="http://schemas.android.com/apk/res
4	/android"
5	
6	xmlns:app="http://schemas.android.com/apk/res-
7	auto"
8	
9	xmlns:tools="http://schemas.android.com/tools"
10	android:id="@+id/main"
11	android:layout_width="match_parent"

12	android:layout_height="match_parent"
13	tools:context=".MainActivity">
14	
15	<FrameLayout
16	android:id="@+id/navigation_content"
17	android:layout_width="match_parent"
18	android:layout_height="match_parent"/>
19	
20	<com.google.android.material.bottomnavigation.BottomNavigationView
	android:layout_width="match_parent"
	android:layout_height="wrap_content"
	android:id="@+id/btn_navigation"
	android:background="@color/navi_bottom"
	app:menu="@menu/bottom_navigation_menu"
	android:layout_alignParentBottom="true"
	/>
	</RelativeLayout>

9. ListCharacterAdapter.kt

Tabel 9. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.adapter
3	ter
4	
5	import android.content.Context
6	import android.view.LayoutInflater
7	import android.view.ViewGroup
8	import androidx.recyclerview.widget.RecyclerView
9	import com.bumptech.glide.Glide

```

10 import
11 com.bumptechn.glide.load.resource.bitmap.RoundedCo
12 rners
13 import com.bumptechn.glide.request.RequestOptions
14 import
15 com.example.modul3scrollablelistwarikmaxxing.mode
16 ls.Character
17 import
18 com.example.modul3scrollablelistwarikmaxxing.data
19 binding.ItemCharacterBinding
20
    class ListCharacterAdapter(
        private val context: Context,
        private val listCharacter:
ArrayList<Character>,
        private val onDetailsClick:(String, String,
Int, String) -> Unit,
        private val onInstagramClick:(String) -> Unit
    ):
RecyclerView.Adapter<ListCharacterAdapter.DataVie
wHolder>() {

        inner class DataViewHolder (val binding:
ItemCharacterBinding) :
RecyclerView.ViewHolder(binding.root)

        override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): DataViewHolder {
            val binding =
ItemCharacterBinding.inflate(LayoutInflater.from(
parent.context), parent, false)
            return DataViewHolder(binding)

```

```

    }

    override fun onBindViewHolder(holder:
DataViewHolder, position: Int) {
        with(holder) {
            with(listCharacter[position]) {
                binding.tvItemName.text =
this.name
                binding.tvItemWarikName.text =
this.warikName
                Glide.with(context)
                    .load(this.photo)

                .apply(RequestOptions().transform(RoundedCorners(
15)))

                    .into(binding.imgItemPhoto)
            //
            binding.imgItemPhoto.setImageResource(this.photo)

            binding.btnGotoInstagram.setOnClickListener{

                onInstagramClick(this.linkInstagram)
            }

            binding.btnDetails.setOnClickListener {
                onDetailsClick(this.name,
this.warikName, this.photo, this.details)
            }
        }
    }
}

```

	<pre> override fun getItemCount(): Int = listCharacter.size fun setData (newList: List<Character>) { listCharacter.clear() listCharacter.addAll(newList) } } </pre>
--	--

10. HomeViewModel.kt

Tabel 10. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.pres
3	entation.home
4	
5	import android.annotation.SuppressLint
6	import android.content.res.Resources
7	import androidx.lifecycle.ViewModel
8	import androidx.lifecycle.viewModelScope
9	import
10	com.example.modul3scrollablelistwarikmaxxing.R
11	import
12	com.example.modul3scrollablelistwarikmaxxing.mode
13	ls.Character
14	import kotlinx.coroutines.flow.Flow
15	import kotlinx.coroutines.flow.MutableStateFlow
16	import kotlinx.coroutines.flow.StateFlow
17	import kotlinx.coroutines.flow.flow
18	import kotlinx.coroutines.flow.onStart
19	import kotlinx.coroutines.launch
20	
	class HomeViewModel(private val resources:

```

Resources) : ViewModel() {
    // MutableStateFlow untuk menyimpan list
item(private)
    private val _characterList =
MutableStateFlow<List<Character>>(emptyList())

    // public
    val characterList: StateFlow<List<Character>>
get() = _characterList

    // fungsi yang mengembalikan data list
character
    @SuppressWarnings("Recycle")
    private fun getCharacterFlow():
Flow<List<Character>> = flow {
        // mengambil data dari resources
        val dataName =
resources.getStringArray(R.array.data_name)
        val dataWarikName =
resources.getStringArray(R.array.data_warik_name)
        val dataLinkInstagram =
resources.getStringArray(R.array.data_link)
        val dataPhoto =
resources.obtainTypedArray(R.array.data_photo)
        val dataDetails =
resources.getStringArray(R.array.data_details)

        // membuat list
        val listCharacter =
ArrayList<Character>()
        for (i in dataName.indices) {
            val character =

```

```

Character(dataName[i], dataWarikName[i],
dataLinkInstagram[i], dataPhoto.getResourceId(i,
-1), dataDetails[i])

        listCharacter.add(character)
    }
    // untuk free up memory
    dataPhoto.recycle()
    // mengirim (emit) item ke collector
    emit(listCharacter)
}

fun loadCharacters() {
    // jalankan coroutine
    viewModelScope.launch {
        getCharacterFlow()
            .onStart {
                _characterList.value =
emptyList()
            }
            .collect { characters ->
                // saat data berhasil emit
oleh flow, update StateFlow
                _characterList.value =
characters
            }
    }
}
}

```

11. HomeViewModelFactory.kt

Tabel 11. Source Code Jawaban Soal

1	package
2	com.example.modul3scrollablelistwarikmaxxing.pres
3	entation.home
4	
5	import android.annotation.SuppressLint
6	import android.content.res.Resources
7	import androidx.lifecycle.ViewModel
8	import androidx.lifecycle.viewModelScope
9	import
10	com.example.modul3scrollablelistwarikmaxxing.R
11	import
12	com.example.modul3scrollablelistwarikmaxxing.mode
13	ls.Character
14	import kotlinx.coroutines.flow.Flow
15	import kotlinx.coroutines.flow.MutableStateFlow
16	import kotlinx.coroutines.flow.StateFlow
17	import kotlinx.coroutines.flow.flow
18	import kotlinx.coroutines.flow.onStart
19	import kotlinx.coroutines.launch
20	
	class HomeViewModel(private val resources:
	Resources) : ViewModel() {
	// MutableStateFlow untuk menyimpan list
	item(private)
	private val _characterList =
	MutableStateFlow<List<Character>>(emptyList())
	// public
	val characterList: StateFlow<List<Character>>
	get() = _characterList

```

        // fungsi yang mengembalikan data list
        character

        @SuppressLint("Recycle")
        private fun getCharacterFlow():
        Flow<List<Character>> = flow {
            // mengambil data dari resources
            val dataName =
            resources.getStringArray(R.array.data_name)
            val dataWarikName =
            resources.getStringArray(R.array.data_warik_name)
            val dataLinkInstagram =
            resources.getStringArray(R.array.data_link)
            val dataPhoto =
            resources.obtainTypedArray(R.array.data_photo)
            val dataDetails =
            resources.getStringArray(R.array.data_details)

            // membuat list
            val listCharacter =
            ArrayList<Character>()
            for (i in dataName.indices) {
                val character =
                Character(dataName[i], dataWarikName[i],
                dataLinkInstagram[i], dataPhoto.getResourceId(i,
                -1), dataDetails[i])
                listCharacter.add(character)
            }
            // untuk free up memory
            dataPhoto.recycle()
            // mengirim (emit) item ke collector
            emit(listCharacter)
        }

```

```

        fun loadCharacters() {
            // jalankan coroutine
            viewModelScope.launch {
                getCharacterFlow()
                    .onStart {
                        _characterList.value =
emptyList()
                    }
                    .collect { characters ->
                        // saat data berhasil emit
oleh flow, update StateFlow
                        _characterList.value =
characters
                    }
                }
            }
        }
    }
}

```

12. HomeFragment.kt

Tabel 12. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.pres
3	entation.home
4	
5	import android.annotation.SuppressLint
6	import android.content.Intent
7	import android.net.Uri
8	import android.os.Bundle
9	import android.util.Log
10	import androidx.fragment.app.Fragment
11	import android.view.LayoutInflater
12	import android.view.View
13	import android.view.ViewGroup
14	import androidx.fragment.app.viewModels
15	import androidx.lifecycle.lifecycleScope
16	import
17	androidx.recyclerview.widget.LinearLayoutManager
18	import
19	com.example.modul3scrollablelistwarikmaxxing.R
20	import
	com.example.modul3scrollablelistwarikmaxxing.adap
	ter.ListCharacterAdapter
	import
	com.example.modul3scrollablelistwarikmaxxing.data
	binding.FragmentHomeBinding
	import
	com.example.modul3scrollablelistwarikmaxxing.mode
	ls.Character
	import
	com.example.modul3scrollablelistwarikmaxxing.pres

```

entation.detail.DetailFragment
import
com.example.modul3scrollablelistwarikmaxxing.util
s.HomeViewModelFactory
import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.launch

class HomeFragment : Fragment() {

    private var _binding: FragmentHomeBinding? =
null
    private val binding get() = _binding!!

    private lateinit var characterAdapter:
ListAdapter

    private val viewModel: HomeViewModel by
viewModel {
        HomeViewModelFactory(resources)
    }

    override fun onCreateView(
        inflater: LayoutInflater, container:
ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        _binding =
FragmentHomeBinding.inflate(inflater, container,
false)
        return binding.root
    }

```

```

        override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
            super.onViewCreated(view,
savedInstanceState)

            setupRecyclerView()
            observeCharacterList()

            viewModel.loadCharacters()
        }

        private fun setupRecyclerView () {
            characterAdapter = ListCharacterAdapter(
                requireContext(),
                ArrayList(),
                onInstagramClick = { link ->
                    Log.e("Intent to Instagram",
"Going to $link")

                    val intent =
Intent(Intent.ACTION_VIEW, Uri.parse(link))
                    startActivity(intent)
                },
                onDetailsClick = { name, warikName,
photo, details ->
                    val detailFragment =
DetailFragment().apply {
                        Log.e("Move to detail page",
"move to $name detail page")

                        arguments = Bundle().apply {

```

```

        putString("EXTRA_NAME",
name)

        putString("EXTRA_NAME2",
warikName)

        putInt("EXTRA_PHOTO",
photo)

putString("EXTRA_DETAILS", details)
    }
}

parentFragmentManager.beginTransaction()
    .replace(R.id.main,
detailFragment)
    .addToBackStack(null)
    .commit()
}
)

binding.rcCharacter.apply {
    layoutManager =
LinearLayoutManager(context)
    adapter = characterAdapter
    setHasFixedSize(true)
}
}

private fun observeCharacterList() {
    viewLifecycleOwner.lifecycleScope.launch
{
        viewModel.characterList.collectLatest

```

	<pre> { list -> characterAdapter.setData(list) } } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	--

13. DetailFragment.kt

Tabel 13. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.pres
3	entation.detail
4	
5	import android.os.Bundle
6	import androidx.fragment.app.Fragment
7	import android.view.LayoutInflater
8	import android.view.View
9	import android.view.ViewGroup
10	import com.bumptech.glide.Glide
11	import
12	com.example.modul3scrollablelistwarikmaxxing.data
13	binding.FragmentDetailBinding
14	
15	class DetailFragment : Fragment() {
16	private var _binding: FragmentDetailBinding?
17	= null
18	private val binding get() = _binding!!

19	
20	<pre> override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View? { // Inflate the layout for this fragment _binding = FragmentDetailBinding.inflate(inflater, container, false) val name = arguments?.getString("EXTRA_NAME") val warikName = arguments?.getString("EXTRA_NAME2") val photo = arguments?.getInt("EXTRA_PHOTO") val details = arguments?.getString("EXTRA_DETAILS") binding.tvName.text = name binding.tvWarikName.text = warikName binding.tvDetails.text = details // glide for image Glide.with(this) .load(photo) .circleCrop() .into(binding.imgItemPhoto) return binding.root } </pre>

	<pre> override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	---

14. InfoFragment.kt

Tabel 14. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing.pres
3	entation.info
4	
5	import android.os.Bundle
6	import androidx.fragment.app.Fragment
7	import android.view.LayoutInflater
8	import android.view.View
9	import android.view.ViewGroup
10	import com.bumptech.glide.Glide
11	import
12	com.bumptech.glide.load.resource.bitmap.RoundedCo
13	rn timers
14	import com.bumptech.glide.request.RequestOptions
15	import
16	com.example.modul3scrollablelistwarikmaxxing.R
17	import
18	com.example.modul3scrollablelistwarikmaxxing.data
19	binding.FragmentInfoBinding
20	
	<pre> class InfoFragment : Fragment() { private lateinit var binding: FragmentInfoBinding </pre>

	<pre> override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?): View? { // Inflate the layout for this fragment binding = FragmentInfoBinding.inflate(inflater, container, false) // glide image Glide.with(this) .load(R.drawable.img) .apply(RequestOptions().transform(RoundedCorners(25))) .into(binding.imgItemPhotoWarik) return binding.root } </pre>
--	--

15. MainActivity.kt

Tabel 15. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing
3	
4	import android.os.Bundle
5	import android.util.Log
6	import android.view.Menu
7	import android.view.MenuItem
8	import androidx.activity.enableEdgeToEdge

9	<code>import androidx.appcompat.app.AppCompatActivity</code>
10	<code>import androidx.fragment.app.Fragment</code>
11	<code>import</code>
12	<code>com.example.modul3scrollablelistwarikmaxxing.data</code>
13	<code>binding.ActivityMainBinding</code>
14	<code>import</code>
15	<code>com.example.modul3scrollablelistwarikmaxxing.pres</code>
16	<code>entation.home.HomeFragment</code>
17	<code>import</code>
18	<code>com.example.modul3scrollablelistwarikmaxxing.pres</code>
19	<code>entation.info.InfoFragment</code>
20	<code>class MainActivity : AppCompatActivity() {</code> <code> private lateinit var binding:</code> <code>ActivityMainBinding</code> <code> val fragHome = HomeFragment()</code> <code> val fragInfo = InfoFragment()</code> <code> val mFragmentManager = <i>supportFragmentManager</i></code> <code> // active fragment</code> <code> var active: Fragment = fragHome</code> <code> private lateinit var menu: Menu</code> <code> private lateinit var menuItem: MenuItem</code> <code> override fun onCreate(savedInstanceState:</code> <code>Bundle?) {</code> <code> super.onCreate(savedInstanceState)</code> <code> enableEdgeToEdge()</code> <code> binding =</code> <code>ActivityMainBinding.inflate(layoutInflater)</code> <code> setContentView(binding.root)</code> <code> }</code> <code>}</code>

	<pre> setUpNaviBottom() } private fun setUpNaviBottom () { mFragmentManager.beginTransaction().<i>apply</i> { add(R.id.navigation_content, fragInfo).hide(fragInfo) add(R.id.navigation_content, fragHome) }.commit() active = fragHome menu = binding.btnNavigation.menu menuItem = menu.getItem(0) menuItem.<i>isChecked</i> = true binding.btnNavigation.setOnNavigationItemSelectedListener Listener { item -> when (item.itemId) { R.id.navigation_home -> { callFrag(0, fragHome) } R.id.navigation_info -> { callFrag(1, fragInfo) } } true } </pre>
--	---

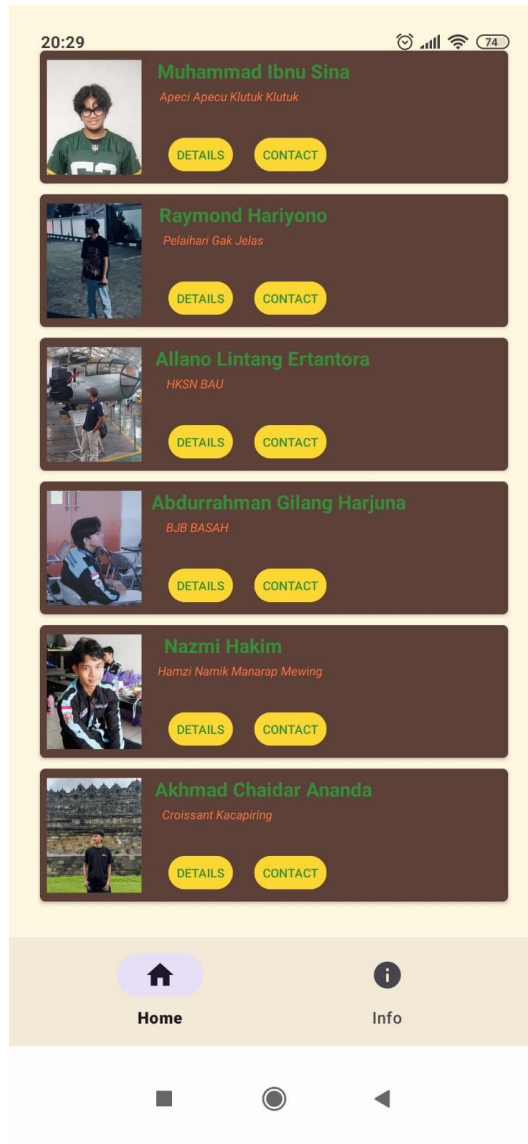
	<pre> } private fun callFrag (i: Int, fragment: Fragment) { if (fragment != active) { Log.d("MyFlexibleFragment", "Fragment Name : " + active::class.java.simpleName) menuItem = menu.getItem(i) menuItem.isChecked = true mFragmentManager.beginTransaction() .hide(active) .show(fragment) .commit() active = fragment } } } </pre>
--	--

16. AppGlideModule.kt

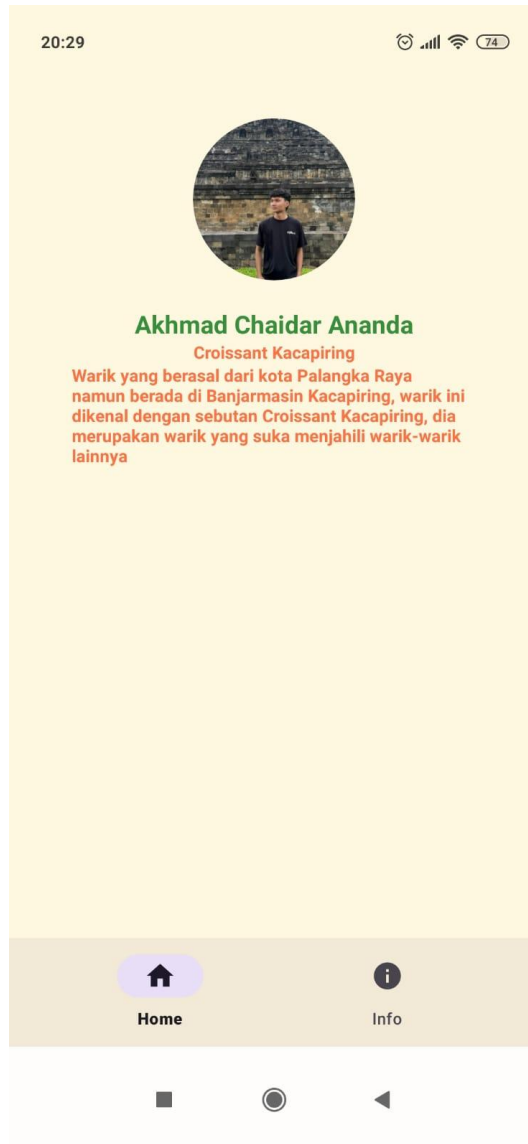
Tabel 16. Source Code Jawaban Soal 1

1	package
2	com.example.modul3scrollablelistwarikmaxxing
3	
4	import com.bumptech.glide.annotation.GlideModule
5	import com.bumptech.glide.module.AppGlideModule
6	
7	@GlideModule
8	class AppGlideModule : AppGlideModule()

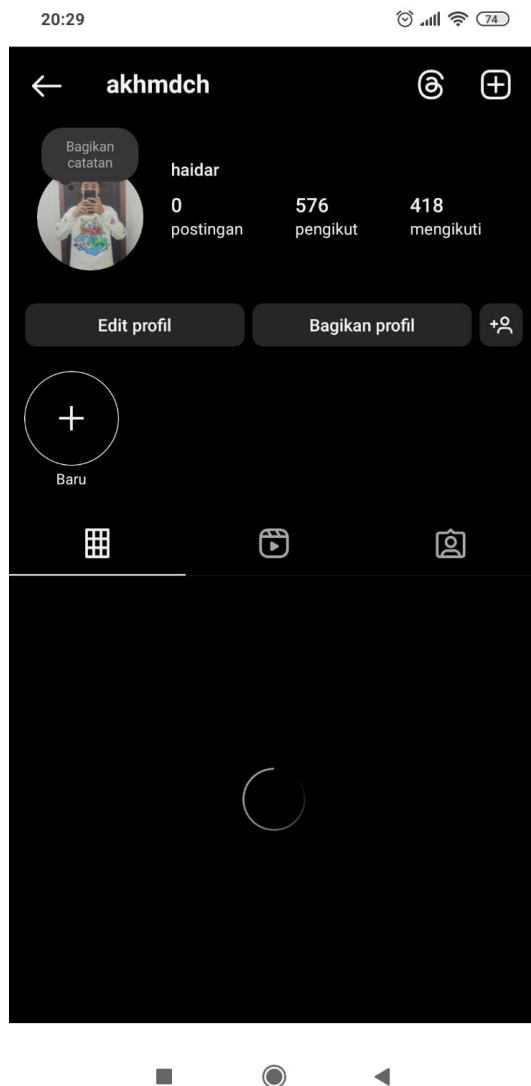
B. Output Program



Gambar 1. Screenshot Output Soal 1



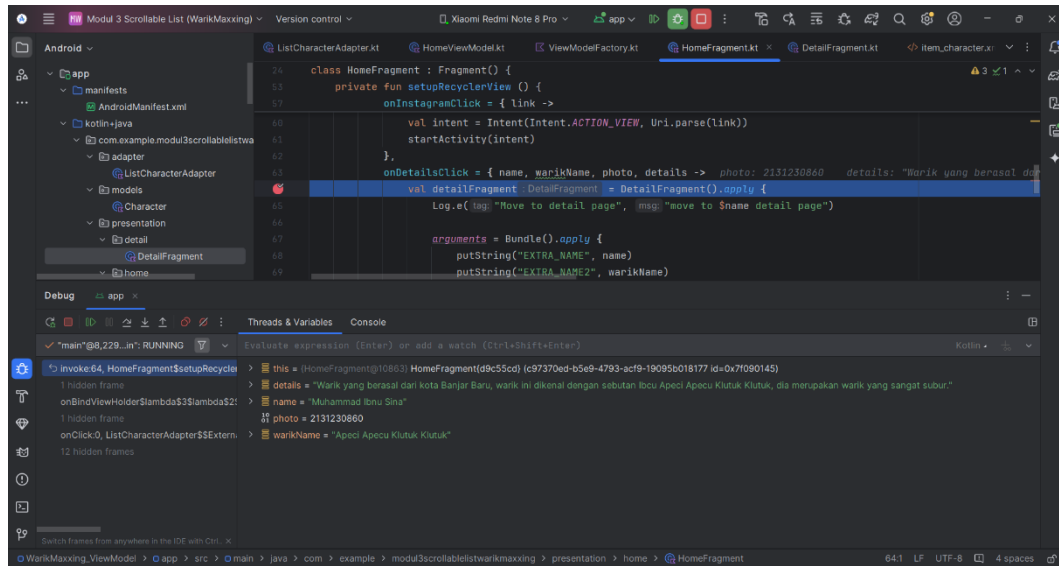
Gambar 2. Screenshot Output Soal 1



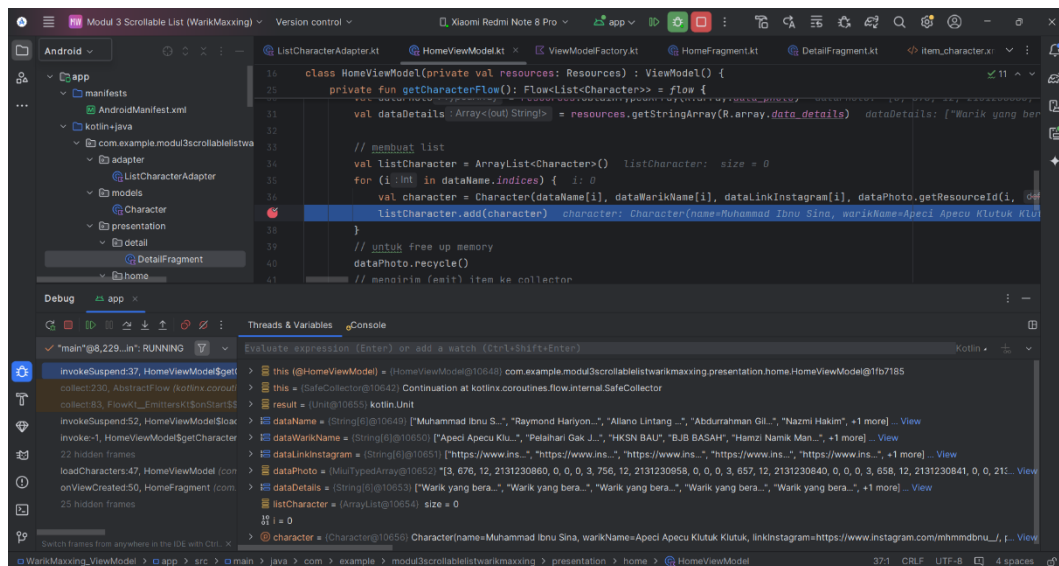
Gambar 3. Screenshot Output Soal 1



Gambar 4. Screenshot Output Soal 1



Gambar 5. Screenshot Hasil Debugging



Gambar 6. Screenshot Hasil Debugging

C. Pembahasan

Aplikasi yang berisikan daftar karakter grup mahasiswa yang bernama *WarikMaxxing*.

1. character.kt

Model data dengan nama *Character* yang digunakan untuk merepresentasikan objek karakter dalam aplikasi. Model ini juga mendukung proses pengiriman data antar komponen Android dengan

menggunakan Parcelable. @Parcelize merupakan anotasi dari Kotlin untuk mengirim objek Character melalui Intent atau Bundle antar Activity atau Fragment.

2. item_character.xml

Tampilan RecyclerView dibuat dalam item_character.xml, dengan menggunakan CardView dan ConstraintLayout. Dalam tampilan, terdapat 5 item, gambar, judul, deskripsi singkat, dan dua tombol (*details* dan *contact*)

Layout utama menggunakan CardView dengan tampilan melengkung untuk memberikan tampilan seperti kartu.

a. CardView

Pembungkus utama item agar terlihat seperti kartu

b. ConstraintLayout

Layout fleksibel untuk mengatur posisi elemen di dalam CardView.

c. ImageView

Menampilkan foto karakter

d. TextView

Menampilkan nama atau julukan deskripsi tentang karakter.

e. MaterialButton (Details dan Contact)

Menampilkan detail karakter yang berpindah ke DetailFragment dan mengarahkan pengguna ke link Instagram karakter.

3. fragment_home.xml

Tampilan RecyclerView yang telah dibuat dalam item_character.xml akan ditampilkan di dalam fragment_home.xml.

Layout utama menggunakan ConstraintLayout, di dalamnya terdapat RecyclerView.

a. RecyclerView

Menampilkan daftar dari layout item_character.xml.

4. fragment_detail.xml

Tampilan halaman detail, layout utama dibuat dengan ConstraintLayout.

a. ImageView

Menampilkan foto karakter, yang akan dibuat menjadi circleCrop dengan menggunakan Glide.

b. TextView (Nama, Julukan, dan Deskripsi detail)

Menampilkan nama, julukan, dan deskripsi tentang karakter.

5. fragment_info.xml

Tampilan halaman info yang berisi informasi mengenai warikmaxxing. Layout utama dibuat dengan LinearLayout dengan tujuan menampilkan informasi secara vertical.

a. ImageView

Menampilkan foto karakter, yang akan dibuat menjadi roundedCorners menggunakan Glide.

b. TextView (Judul dan Deskripsi)

Menampilkan nama dan deskripsi tentang warikmaxxing.

6. bottom_navigation_menu.xml

File yang digunakan sebagai item menu navigasi yang berada di bawah. Menu ini ditampilkan di bagian bawah layar dan memungkinkan pengguna untuk berpindah halaman Home dan Info (*fragment*). File ini juga merupakan penghubung dengan komponen lain yang digunakan oleh MainActivity.

a. Menu

Menu merupakan root dari file yang menampung navigasi antar halaman.

b. Item (Home dan Info)

Kedua item yaitu Home dan Info akan ditampilkan di bawah dan ikon menu ditampilkan menggunakan icon gambar yang diambil dari drawable.

7. **navigation.xml**

File yang menjadi *Navigation Component*, digunakan sebagai alur navigasi antar fragment dalam aplikasi. *Navigation graph* ini menentukan fragment mana yang akan ditampilkan ketika aplikasi pertama kali dibuka, serta fragment lainnya yang dapat dituju oleh user.

Strukturnya menggunakan navigation dan fragment.

a. Navigation

Destinasi pertama yang akan ditampilkan yaitu halaman Home.

b. Fragment (Home dan Info)

Halaman Home dan Info dipanggil agar dapat diakses melalui navigasi.

8. **activity_main.xml**

File yang digunakan sebagai tampilan utama, lalu mengatur posisi tiap fragment dan menampilkan navigasi yang berada di bawah layar sebagai navigasi utama aplikasi.

Layout utama menggunakan `RelativeLayout` sebagai root yang mengatur posisi relatif tiap elemen di dalamnya. Lalu menggunakan `FrameLayout` sebagai container tiap fragment (Home dan Info). Kemudian yang terakhir, menampilkan komponen dari `MaterialDesign` untuk tampilan navigasi bawah.

9. `ListCharacterAdapter.kt`

Adapter khusus untuk menyimpan daftar karakter dan menampilkan data karakter ke dalam bentuk `RecyclerView` di halaman utama aplikasi.

a. Kelas

```
• class ListCharacterAdapter(  
    private val context: Context,  
    private val listCharacter:  
    ArrayList<Character>,  
    private val onDetailsClick: (String, String,  
    Int, String) -> Unit,  
    private val onInstagramClick: (String) ->  
    Unit  
)
```

Adapter yang menerima *context* sebagai pemanggilan *resource* gambar, *listCharacter* yang menyimpan data karakter, *onDetailsClick* dan *onInstagramClick* sebagai *callback* untuk tombol.

b. ViewHolder

```
• inner class DataViewHolder  
    (val binding: ItemCharacterBinding) :  
    RecyclerView.ViewHolder(binding.root)
```

Terdapat adapter yang mengatur tampilan data karakter dan list Array yang menampung data karakternya.

c. `onCreateViewHolder()`

- `val binding =
ItemCharacterBinding.inflate(LayoutInflater.from
(parent.context), parent, false)`

Melakukan inflate layout untuk setiap item list karakter.

d. `onBindViewHolder()`

Fungsi ini bertujuan untuk membuat *instance* dari adapter dan mengatur 2 fungsi lambda, yang khusus ke tombol detail dan contact.

- `with(listCharacter[position]) {
binding.tvItemName.text = this.name
binding.tvItemWarikName.text =
this.warikName
Glide.with(context)
.load(this.photo)

.apply(RequestOptions().transform(RoundedCorners
(15)))
.into(binding.imgItemPhoto)
}`

Menampilkan nama karakter, julukan, dan foto dari tiap karakter. Gambar yang ditampilkan diformat menggunakan Glide dengan tampilan `roundedCorner`.

e. Event Button

- `binding.btnGotoInstagram.setOnClickListener {
onInstagramClick(this.linkInstagram)
}`
- `binding.btnDetails.setOnClickListener {
onDetailsClick(this.name, this.warikName,
this.photo, this.details)
}`

Saat kedua tombol diklik, tombol pertama akan menghubungkan ke halaman yang berisikan detail deskripsi dari tiap karakter,

sedangkan tombol kedua akan mengarahkan pengguna menuju Instagram melalui link.

f. getItemCount()

Mengembalikan jumlah item yang tersimpan yang akan ditampilkan di dalam RecyclerView().

g. setData()

Fungsi yang memperbarui data karakter dalam adapter dengan daftar baru dan selalu memperbaru atau *refresh* RecyclerView.

10. HomeViewModel.kt

File yang yang digunakan untuk mengelola dan menyimpan data karakter dalam arsitektur MVVM, ViewModel ini menyediakan data ke UI menggunakan StateFlow dan Coroutine Flow.

a. Kelas

```
• class HomeViewModel(private val resources:
    Resources) : ViewModel()
```

Resources digunakan untuk mengakses data dari string-array yang ada di values, semua data yang ada di dalamnya.

ViewModel, kelas dari Android Jetpack yang digunakan untuk mengelola UI dari tiap data.

b. StateManagement

```
• private val _characterList =
    MutableStateFlow<List<Character>>(emptyList())
    val characterList: StateFlow<List<Character>>
    get() = _characterList
```

_characterList digunakan untuk untuk menyimpan list karakter dan characterList merupakan data yang dapat digunakan oleh UI agar bisa merespons perubahan data secara otomatis.

c. `getCharacterFlow()`

- ```
private fun getCharacterFlow():
 Flow<List<Character>> = flow {
 val dataName =
 resources.getStringArray(R.array.data_name)

 ...
 val listCharacter = ArrayList<Character>()
 for (i in dataName.indices) {
 ...
 listCharacter.add(character)
 }
 dataPhoto.recycle()
 emit(listCharacter)
 }
}
```

Fungsi ini membaca data dari tiap karakter yang ada di resources array. Semua data tersebut dikumpulkan dan flow akan meng-emit list tersebut. Emit digunakan untuk menghasilkan data yang telah dikumpulkan untuk dialirkan ke dalam suatu aliran data.

d. `loadCharacters()`

- ```
fun loadCharacters() {  
    viewModelScope.launch {  
        getCharacterFlow()  
            .onStart { _characterList.value =  
emptyList() }  
            .collect { characters ->  
                _characterList.value =  
characters  
            }  
    }  
}
```

Fungsi ini menjalankan proses pengambilan data secara asinkron dengan menggunakan coroutine. Lalu menggunakan viewModelScope untuk menjamin coroutine aktif saat ViewModel hidup saja. Onstart() dan collect digunakan untuk mengatur data

kosong sebelum data baru dimuat dan hasilnya ditangkap lalu diperbarui StateFlow.

11. HomeViewModelFactory.kt

File yang digunakan untuk membuat instance HomeViewModel yang telah dibuat sebelumnya. HomeViewModel sebelumnya memerlukan parameter eksternal yaitu resources.

a. Kelas

- ```
class HomeViewModelFactory(private val
resources: Resources) :
 ViewModelProvider.Factory
```

Resources digunakan untuk mengakses data dari string-array yang ada di values, semua data yang ada di dalamnya.

Fungsi ini mewarisi ViewModelProvider yang merupakan antarmuka untuk membuat ViewModel secara manual

### b. create()

- ```
override fun <T : ViewModel> create(modelClass:
Class<T>): T {
    if
    (modelClass.isAssignableFrom(HomeViewModel::clas
s.java)) {
        @Suppress("UNCHECKED_CAST")
        return HomeViewModel(resources) as T
    }
    throw IllegalArgumentException("Unknown
ViewModel Class")
}
```

Fungsi ini akan dipanggil ketika ViewModel ingin dibuat, lalu mengecek apakah modelClass sesuai dengan HomeViewModel yang dibuat. Jika ya, HomeViewModel akan dibuat secara manual dengan menyertakan resources yang telah diambil. Jika tidak, program akan melempar exception yaitu ViewModel tidak diketahui.

12. HomeFragment.kt

Tampilan utama aplikasi yang menampilkan daftar karakter dengan menggunakan RecyclerView, fragment ini bertanggung jawab dalam menampilkan daftar dalam bentuk scrollable list dan menangani event klik pada tombol detail dan contact.

a. ViewBinding

- `private var _binding: FragmentHomeBinding? = null`
- `private val binding get() = _binding!!`

ViewBinding digunakan agar mempermudah akses view ke tampilan `fragment_home.xml`.

b. List dan Adapter

- `private lateinit var characterAdapter: ListCharacterAdapter`
- `private val list = ArrayList<Character>()`

Terdapat adapter yang mengatur tampilan data karakter dan list Array yang menampung data karakternya.

c. onCreateView

- `list.clear()`
- `list.addAll(getListCharacter())`
- `setupRecyclerView()`

List akan dibersihkan setiap kali dipanggil dan menambahkan data tiap karakternya. Lalu juga melakukan setup dengan pemanggilan fungsi `setupRecyclerView()`.

d. setupRecylcerView()

Fungsi ini bertujuan untuk membuat *instance* dari adapter dan mengatur 2 fungsi lambda, yang khusus ke tombol detail dan contact.

- `onInstagramClick:`
Membuka link Instagram menggunakan `Intent.ACTION_VIEW`.
- `onDetailsClick:`
Membuka `DetailFragment` dan mengirim data karakter lewat `Bundle`.

Kemudian melakukan konfigurasi `RecyclerView`:

- `binding.rcCharacter.apply {...}`

e. `getListCharacter()`

Fungsi yang mengambil data dari data pada folder `strings.xml` untuk membentuk list dari objek `Character`.

f. `onDestroyView()`

Fungsi yang mencegah memory leak dengan menghapus referensi binding saat view dihancurkan.

13. `DetailFragment.kt`

Tampilan yang menampilkan detail informasi dari tiap karakter ketika pengguna mengklik salah satu item pada `HomeFragment` yaitu halaman utama aplikasi.

a. `ViewBinding`

- `private var _binding: FragmentDetailBinding? = null`
- `private val binding get() = _binding!!`

`ViewBinding` digunakan agar mempermudah akses view ke tampilan `fragment_detail.xml`.

b. `onCreateView`

- `binding` =
`FragmentDetailBinding.inflate(inflater,`
`container, false)`

Melakukan inflate terhadap layout `fragment_detail.xml` agar tampilan terpanggil.

c. Pengambilan Data melalui Argument

- `val name = arguments?.getString("EXTRA_NAME")`
- `val warikName =`
`arguments?.getString("EXTRA_NAME2")`
- `val photo = arguments?.getInt("EXTRA_PHOTO")`
- `val` `details` =
`arguments?.getString("EXTRA_DETAILS")`

Data dikirim dari `HomeFragment` ke `DetailFragment` melalui `Bundle` dan diambil dengan fungsi `getArguments()` yang bertipe data string dan integer.

Setelah data diterima melalui argumen, data tersebut akan di-set ke view menggunakan binding, menampilkan nama, julukan, foto, dan informasi detail dari tiap karakter.

- `binding.tvName.text = name`
- `binding.tvWarikName.text = warikName`
- `binding.tvDetails.text = details`

Foto ditampilkan dengan menggunakan library `Glide` untuk memuat gambar secara langsung dari `drawable` dan membuat foto dalam bentuk lingkaran seperti foto profil.

d. `onDestroyView()`

Fungsi yang mencegah memory leak dengan menghapus referensi binding saat view dihancurkan.

14. `InfoFragment.kt`

Tampilan opsional yang memberi tahu pengguna informasi mengenai aplikasi yang dibuat.

a. **ViewBinding**

- `private lateinit var binding: FragmentInfoBinding`

ViewBinding digunakan agar mempermudah akses view ke tampilan `fragment_info.xml`.

b. **onCreateView**

- `binding =
FragmentDetailBinding.inflate(inflater,
container, false)`

Melakukan inflate terhadap layout `fragment_detail.xml` agar tampilan terpanggil.

Foto ditampilkan dengan menggunakan library Glide untuk memuat gambar secara langsung dari drawable dan membuat foto berbentuk rounded.

15. AppGlideModule.kt

File yang digunakan untuk mengkonfigurasi Glide secara global.

16. MainActivity.kt

Kelas utama dalam aplikasi yang bertanggung jawab untuk menghubungkan tiap fragment, tampilan utama aplikasi, navigasi antar fragment (tampilan navigasi di bawah aplikasi), menampilkan dua halaman (Home dan Info), dan menjaga hanya satu fragment yang ditampilkan secara aktif.

- `val fragHome = HomeFragment()`
- `val fragInfo = InfoFragment()`
- `var active: Fragment = fragHome`

Inisialisasi fragment dengan membuat dua fragment utama dan menyimpan fragment yang sedang aktif agar tidak direplace ulang.

- `mFragmentManager.beginTransaction().apply {
 add(R.id.navigation_content,
 fragInfo).hide(fragInfo)
 add(R.id.navigation_content, fragHome)
}.commit()`

Fungsi `setUpNaviBottom()` yang berguna untuk penambahan kedua halaman ke dalam navigasi, tetapi hanya halaman home yang langsung terlihat.

- `binding.btnCalculate.setOnNavigationClickListener {
}`

Menangani event ketika item navigasi ditekan dan memanggil fungsi `callFrag()` untuk menampilkan halaman yang sesuai.

- `if (fragment != active) {
 mFragmentManager.beginTransaction()
 .hide(active)
 .show(fragment)
 .commit()
 active = fragment
}`

Fungsi ini bertujuan untuk menyembunyikan fragment yang sedang aktif agar tidak bertabrakan, lalu menampilkan fragment yang sesuai dengan fragment yang dipilih melalui bagian navigasi.

17. Tool Debugger

Debugger merupakan tools untuk menjalankan aplikasi secara perlahan (step by step), agar user bisa:

- Melihat isi variabel dan state saat *runtime*.
- Menemukan bug atau perilaku pada aplikasi yang tidak sesuai harapan.
- Menguji logika program tanpa harus menjalankan program atau menggunakan Log.d atau Toast.

Pada debugger terdapat tombol pada *debug* di bagian atas untuk running fitur debug dan terdapat sebuah informasi debug pada bagian kiri. Tombol atau fitur-fitur yang tersedia seperti:

- Resume Program

Melanjutkan eksekusi dalam program.

- Step Into

Jika kita ingin melihat isi dari fungsi tersebut, kita akan menggunakan step into untuk masuk ke dalam fungsi yang sedang dipanggil.

- Step Over

Jika kita ingin tahu apakah fungsi yang kita mau atau breakpoint bekerja dengan benar dan ingin lanjut ke baris selanjutnya. Fungsi fitur ini untuk melewati fungsi tanpa masuk ke dalamnya.

- Step Out

Jika kita sudah cukup melihat bagian dalam fungsi, program akan keluar dari fungsi saat ini dan kembali ke fungsi pemanggil.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application class adalah kelas dasar yang berfungsi untuk mempertahankan global application state, yaitu status aplikasi keseluruhan selama siklus dalam aplikasi berjalan.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat

<https://github.com/akhmadCh/Pemrograman-Mobile/tree/master/Modul%204>